

0.91.5. Le snake — une queue qui suit le A

Le 0.91.4, et le A a une queue : un O, toujours sur la case que le A vient de quitter. Le début d'un serpent.

1. Ce qu'il faut comprendre

C'est le 0.91.4, avec une chose en plus : une queue. Derrière le A, il y a un O (`ALPHABET[14]` , la 15e lettre).

Quand le A fait un pas, le O le suit : il prend **la case que le A vient de quitter**. C'est le début d'un jeu de serpent, le « snake ».

Deux nouvelles variables, `qx` et `qy` : la colonne et la ligne de la queue. Au départ, (4, 8) : juste à gauche du A, qui est en (5, 8). `nouvellePartie()` les remet là, et y pose le O.

Où était le A avant son pas ? On le sait déjà : au 0.91.3, on a rangé sa place d'avant dans `ax` et `ay` , pour le faire reculer devant un mur. On s'en sert une deuxième fois : c'est exactement là que la queue doit aller.

Ce qui est nouveau ici : savoir si le A a bougé. `deplace_croix` ne fait un pas que toutes les 250 ms ; les autres images, le A ne bouge pas. Et contre un mur, il est revenu en (`ax` , `ay`). Il a bougé si sa place n'est **plus** celle d'avant : `x != ax || y != ay` . `!=` veut dire « n'est pas égal à » (le 0.82), `||` veut dire « ou » (le 0.91.3) : une seule des deux différences suffit (un pas à gauche ou à droite change `x` , un pas en haut ou en bas change `y`).

Quand il a bougé, quatre étapes, dans cet ordre : 1. `effacer(qx, qy, 1)` efface l'ancien O (le `1` : une seule case). 2. `qx = ax;` et `qy = ay;` : la queue prend la case quittée par le A. 3. `poser(qx, qy, ALPHABET[14])` dessine le O à sa nouvelle place. 4. `poser(x, y, ALPHABET[0])` redessine le A.

Pourquoi redessiner le A ? Pour le demi-tour. Le A est en (5, 8), sa queue en (4, 8). Il va à gauche : il arrive en (4, 8), **sur** sa queue. L'étape 1 efface (4, 8)... et efface donc le A ! L'étape 3 pose le O en (5, 8), la case quittée. L'étape 4 remet le A en (4, 8). Le A et sa queue ont échangé leurs places.

Déroulons un pas à droite : A en (5, 8), O en (4, 8). `deplace_croix` : A en (6, 8), et (5, 8) est effacée. `x` vaut 6, `ax` vaut 5 : il a bougé. 1. (4, 8) est effacée. 2. `qx = 5`, `qy = 8`. 3. O en (5, 8). 4. A en (6, 8). Résultat : **O A**, un cran plus loin.

Encore un défaut, pour plus tard : si le P réapparaît juste sur la queue, le O l'efface au pas suivant ; on ne le voit plus, mais il est toujours là, dans `px` et `py` .

Les fonctions de cette leçon — et le tuto de chacune

- `#include <reste>` → [0.8.4. #include <reste> — a % b](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <couleurTexte>` → [0.87.7. La fonction couleurTexte\(\) — la couleur des lettres](#)
- `#include <texteGrand>` → [0.86.3. La fonction texteGrand\(\) — un texte agrandi](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <deplace_croix>` → [0.66.3. La fonction deplace_croix\(\) — suivre la croix, case par case](#)
- `#include <hasard>` → [0.81.2. La fonction hasard\(\) — un nombre au hasard](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **2463 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	122 o	← la console, à l'allumage — la mise en route de la console
texte_grand_0()	195 o	← main() — ta fonction
chaque_minuteur()	124 o	← main() — une fonction fournie par le langage
deplace_croix()	217 o	← main() — une fonction fournie par le langage
viderEcran()	33 o	← nouvellePartie(), main() — ta fonction
nouvellePartie()	181 o	← main() — ta fonction
main()	413 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte, EcrireNombre — attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), effacer() — trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() — efface des cases
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
EcrireNombre	33 o	← nombre() — écrit un nombre en chiffres
Reste	10 o	← main() — le reste d'une division
CopierTuiles	18 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Hasard	25 o	← hasard() — tire un nombre au hasard
LireManette	37 o	← bouton() — lit les huit boutons
Tuiles	848 o	← CopierTuiles — la police entière, et 9 tuiles dessinées
"SCORE"	5 o	← texte() — un texte gravé
"TEMPS"	5 o	← texte() — un texte gravé
"APPUIE SUR START"	16 o	← texte() — un texte gravé
"FIN"	3 o	← texte() — un texte gravé
"SCORE"	5 o	← texte() — un texte gravé
"START : REJOUER"	15 o	← texte() — un texte gravé
GRAND_TABLE_0_0_0	12 o	← poser() — une table gravée
GRAND_TABLE_0_1_1	12 o	← poser() — une table gravée
GRAND_TABLE_0_2_2	12 o	← poser() — une table gravée
GRAND_TABLE_0_3_3	12 o	← poser() — une table gravée

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, Diviser, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

poser()	× 16	275 o sur place, dans texte_grand_0(), deplace_croix(), nouvellePartie(), main() + AdresseCase (10 o), GRAND_TABLE_0_0_0 (12 o), GRAND_TABLE_0_1_1 (12 o), GRAND_TABLE_0_2_2 (12 o), GRAND_TABLE_0_3_3 (12 o)
texte()	× 6	66 o sur place, dans nouvellePartie(), main() + EcrireTexte (10 o)
effacer()	× 3	44 o sur place, dans deplace_croix(), viderEcran(), main() + AdresseCase (10 o), EffacerCases (12 o)
bouton()	× 6	30 o sur place, dans deplace_croix(), main() + LireManette (37 o)
nombre()	× 3	30 o sur place, dans main() + EcrireNombre (33 o)
couleurFond()	× 1	12 o sur place, dans main()
hasard()	× 2	6 o sur place, dans main() + Hasard (25 o)
images()	× 2	4 o sur place, dans chaque_minuteur(), deplace_croix()
image()	× 1	3 o sur place, dans main() + AttendreImage (28 o)

Tes fonctions appelées :

deplace_croix()	× 1	19 o sur place, dans main()
chaque_minuteur()	× 1	10 o sur place, dans main()
viderEcran()	× 2	6 o sur place, dans nouvellePartie(), main()
nouvellePartie()	× 2	6 o sur place, dans main()
texte_grand_0()	× 1	3 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <reste> // a % b, sauf par 1, 2, 4, 8, 16... écrits en clair
#include <effacer> // efface des cases, ou tout le fond
#include <poser> // pose une tuile sur une case du fond
#include <ALPHABET> // les lettres de la police : ALPHABET[0] est le A
#include <texte> // écrit un texte à l'écran
#include <couleurTexte> // choisit la couleur des lettres
#include <texteGrand> // écrit un texte agrandi, de 1 à 20 fois
#include <bouton> // lit un bouton de la manette
#include <deplace_croix> // une tuile qui suit la croix, case par case
#include <hasard> // tire un nombre au hasard
#include <nombre> // écrit un nombre en chiffres
#include <chaque> // répond 1 toutes les n millisecondes
```

2. UNE VARIABLE — ECRAN

```
uint8_t ecran = 0; // 0 = le TITRE, 1 = le JEU, 2 = la FIN
```

3. UNE VARIABLE — X

```
uint8_t x = 5; // le A : sa colonne...
```

4. UNE VARIABLE — Y

```
uint8_t y = 8; // ...et sa ligne (le 0.81)
```

5. UNE VARIABLE — PX

```
uint8_t px = 15; // la pièce, le P : sa colonne...
```

6. UNE VARIABLE — PY

```
uint8_t py = 8; // ...et sa ligne
```

7. UNE VARIABLE — SCORE

```
uint8_t score = 0; // les pièces ramassées
```

8. UNE VARIABLE — TEMPS

```
uint8_t temps = 30; // les secondes qui restent
```

9. UNE VARIABLE — AX

```
uint8_t ax = 5;           // la place du A AVANT son pas... (le 0.91.3)
```

10. UNE VARIABLE — AY

```
uint8_t ay = 8;           // ...pour l'y remettre s'il entre dans un mur
```

11. UNE VARIABLE — QX

```
uint8_t qx = 4;           // NOUVEAU : la QUEUE, un 0 : sa colonne...
```

12. UNE VARIABLE — QY

```
uint8_t qy = 8;           // ...et sa ligne. Elle part juste à gauche du A.
```

13. UNE FONCTION — VIDERECRAN()

```
void viderEcran() {  
    for (uint8_t l = 0; l < 18; l++) {  
        effacer(0, l, 20);  
    }  
}  
  
// ---- Une partie qui commence, TOUT au départ (le 0.91.2) ----  
// Depuis le titre, ET depuis la fin (START : rejouer).
```

14. UNE FONCTION — NOUVELLEPARTIE()

```
void nouvellePartie() {
    x = 5;                                // le A revient à sa place
    y = 8;
    px = 15;                             // le P revient en (15, 8), d'où qu'il soit
    py = 8;
    ax = 5;                               // et sa place d'avant, la même
    ay = 8;
    qx = 4;                               // NOUVEAU : la queue, juste à gauche du A
    qy = 8;
    score = 0;                            // pas encore de pièce
    temps = 30;                           // 30 secondes de nouveau
    ecran = 1;                            // on est sur le JEU
    viderEcran();
    poser(px, py, ALPHABET[15]);
    poser(qx, qy, ALPHABET[14]);          // NOUVEAU : le O de la queue

    // ---- Le cadre de murs, des X tout autour du terrain (le 0.91.3) ----
    //
    // colonne : 0 1 2 ...                19
    // ligne 0 : X X X X X X ... X X X    le haut : ligne 0, colonnes 0 à 19
    // ligne 1 : X                        X
    // ...      X  A      P  X          les côtés : colonnes 0 et 19
    // ligne 15: X                        X
    // ligne 16: X X X X X X ... X X X    le bas : ligne 16
    // ligne 17: SCORE 000 TEMPS 030     la ligne 17 reste aux nombres
    //
    // ALPHABET[23] : la 24e lettre (on compte depuis 0), le X.
    for (uint8_t c = 0; c < 20; c++) {
        poser(c, 0, ALPHABET[23]);       // le haut
        poser(c, 16, ALPHABET[23]);      // le bas
    }
    for (uint8_t l = 1; l < 16; l++) {
        poser(0, l, ALPHABET[23]);       // le côté gauche
        poser(19, l, ALPHABET[23]);      // le côté droit
    }

    texte(0, 17, "SCORE");
    texte(11, 17, "TEMPS");
}
```

```

int main() {
    couleurTexte(31, 16, 0);
    texteGrand(4, 5, "JEU", 4);
    texte(2, 12, "APPUIE SUR START");

    while (true) {
        image();

        if (ecran == 0 && bouton(START)) {
            nouvellePartie();           // la première partie
        }

        if (ecran == 1) {
            ax = x;                      // on retient où est le A... (le 0.91.3)
            ay = y;
            deplace_croix(x, y, ALPHABET[0], 250); // ...il fait peut-être un pas...

            // ...et s'il est arrivé SUR le cadre (colonne 0 ou 19, ligne 0 ou 16) :
            // 1. on remet le X, que le A venait de recouvrir ;
            // 2. le A revient à sa place d'avant, et s'y pose.
            // || veut dire « OU » : une seule des quatre suffit.
            if (x == 0 || x == 19 || y == 0 || y == 16) {
                poser(x, y, ALPHABET[23]); // le mur revient
                x = ax;                     // le A recule
                y = ay;
                poser(x, y, ALPHABET[0]);
            }

            // ---- NOUVEAU : la queue suit le A ----
            // Le A a-t-il bougé ? Sa place n'est plus celle d'avant (ax, ay).
            // (Pas de pas cette image-ci, ou un mur : il est toujours en (ax, ay).)
            // != veut dire « n'est pas égal », || veut dire « OU ».
            if (x != ax || y != ay) {
                effacer(qx, qy, 1); // 1. l'ancien 0 disparaît (1 : une case)
                qx = ax;            // 2. la queue prend la case
                qy = ay;            // que le A vient de quitter
                poser(qx, qy, ALPHABET[14]); // 3. le 0 s'y dessine (la 15e lettre)
                poser(x, y, ALPHABET[0]); // 4. le A par-dessus : au demi-tour,
            } // l'étape 1 venait de l'effacer

            if (x == px && y == py) { // le A sur le P : ramassée (le 0.81)
                score = score + 1;

                // ---- La pièce réapparaît AU HASARD, mais DANS le cadre (le 0.91.4) ----
                //
                // px = 1 + hasard() % 18;
                //      |   |   |
                //      |   |   +-- % 18 : le reste de la division par 18, de 0 à 17.
                //      |   |   Exemple : hasard() rend 137 ; 137 = 7 × 18 + 11 ;
                //      |   |   le reste est 11.
                //      |   +----- un nombre imprévisible, de 0 à 255 (le 0.81.2)
                //      +----- + 1 : on décale de 0...17 à 1...18 → 11 + 1 = 12.
                //
                // Pourquoi pas % 20 comme au 0.81.2 ? Les colonnes 0 et 19 sont des murs X :
                // la pièce doit rester de 1 à 18. Les lignes 0 et 16 aussi sont des murs :
                // la pièce doit rester de 1 à 15, d'où 1 + hasard() % 15.
                px = 1 + hasard() % 18; // une colonne : 1 à 18
                py = 1 + hasard() % 15; // une ligne : 1 à 15
                poser(px, py, ALPHABET[15]); // le nouveau P apparaît là
            }

            nombre(6, 17, score);
            nombre(17, 17, temps);
        }

        if (ecran == 1 && chaque(1000)) {

```

```

int main() {
couleurTexte(31, 16, 0);
texteGrand(4, 5, "JEU", 4);
texte(2, 12, "APPUIE SUR START");

while (true) {
    image();

    if (ecran == 0 && bouton(START)) {
        nouvellePartie();          // la première partie
    }

    if (ecran == 1) {
        ax = x;                      // on retient où est le A... (le 0.91.3)
        ay = y;
        deplace_croix(x, y, ALPHABET[0], 250); // ...il fait peut-être un pas...

        // ...et s'il est arrivé SUR le cadre (colonne 0 ou 19, ligne 0 ou 16) :
        // 1. on remet le X, que le A venait de recouvrir ;
        // 2. le A revient à sa place d'avant, et s'y pose.
        // || veut dire « OU » : une seule des quatre suffit.
        if (x == 0 || x == 19 || y == 0 || y >= 16) {
            poser(x, y, ALPHABET[23]); // le mur revient
            x = ax;                     // le A recule
            y = ay;
            poser(x, y, ALPHABET[0]);
        }

        // ---- NOUVEAU : la queue suit le A ----
        // Le A a-t-il bougé ? Sa place n'est plus celle d'avant (ax, ay).
        // (Pas de pas cette image-ci, ou un mur : il est toujours en (ax, ay).)
        // != veut dire « n'est pas égal », || veut dire « OU ».
        if (x != ax || y != ay) {
            effacer(qx, qy, 1);          // 1. l'ancien O disparaît (1 : une case)
            qx = ax;                     // 2. la queue prend la case
            qy = ay;                     // que le A vient de quitter
            poser(qx, qy, ALPHABET[14]); // 3. le O s'y dessine (la 15e lettre)
            poser(x, y, ALPHABET[0]);    // 4. le A par-dessus : au demi-tour,
        }                               // l'étape 1 venait de l'effacer
        if (x == px && y == py) {        // le A sur le P : ramassée (le 0.81)
            score = score + 1;

            // ---- La pièce réapparaît AU HASARD, mais DANS le cadre (le 0.91.4) ----
            //
            // px = 1 + hasard() % 18;
            //      |   |   |
            //      |   |   +-- % 18 : le reste de la division par 18, de 0 à 17.
            //      |   |   Exemple : hasard() rend 137 ; 137 = 7 × 18 + 11 ;
            //      |   |   le reste est 11.
            //      |   +----- un nombre imprévisible, de 0 à 255 (le 0.81.2)
            //      +----- + 1 : on décale de 0...17 à 1...18 → 11 + 1 = 12.
            //
            // Pourquoi pas % 20 comme au 0.81.2 ? Les colonnes 0 et 19 sont des murs X :
            // la pièce doit rester de 1 à 18. Les lignes 0 et 16 aussi sont des murs :
            // la pièce doit rester de 1 à 15, d'où 1 + hasard() % 15.
            px = 1 + hasard() % 18;      // une colonne : 1 à 18
            py = 1 + hasard() % 15;      // une ligne : 1 à 15
            poser(px, py, ALPHABET[15]); // le nouveau P apparaît là
        }
        nombre(6, 17, score);
        nombre(17, 17, temps);
    }
}

if (ecran == 1 && chaque(1000)) {

```

```

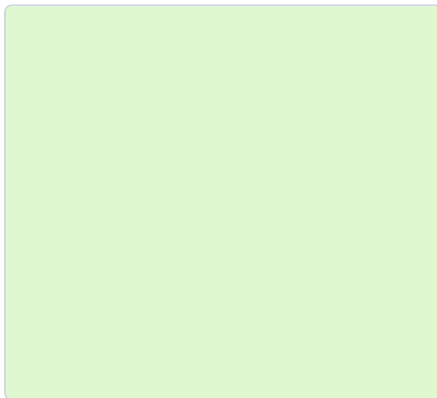
    temps = temps - 1;
    if (temps == 0) {
        ecran = 2;
        viderEcran();
        texte(8, 6, "FIN");
        texte(5, 9, "SCORE");           // le score de la partie (le 0.91.2)
        nombre(11, 9, score);
        texte(2, 13, "START : REJOUER");
    }
}

// ---- Sur la fin, START relance une partie (le 0.91.2).
if (ecran == 2 && bouton(START)) {
    nouvellePartie();                 // la même fonction : tout au départ
}
}
}

```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



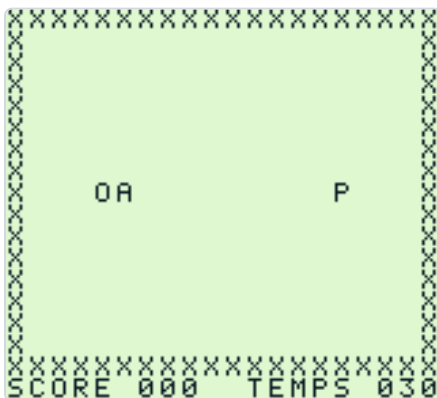
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



START — l'écran a changé

5. Ce qu'on doit voir

START : un O suit le A partout, un cran derrière lui ; au demi-tour, les deux échangent leurs places.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ au départ : O A, en (4, 8) et (5, 8)
- ✓ à droite, le O suit, un cran derrière ($x = 9$)
- ✓ en haut, le O est juste sous le A, et il n'y a qu'un O ($y = 6$)

Ensuite : **0.91.6. Le snake — la queue grandit à chaque pièce** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++